

A Protocol-Aware AI Framework for Securing CPDLC Communications

Cassidy Gorman*, Amir Toliyat*, Maryam Vatankhah†, Christopher Fuller*

* Vaughn College of Aeronautics and Technology

New York, NY, USA

† BMCC, City University of New York

New York, NY, USA

Abstract—This paper presents a protocol-aware AI framework for monitoring Controller–Pilot Data Link Communications (CPDLC) and supporting real-time anomaly detection in aviation operations. The system combines deterministic, contract-based protocol checks with lightweight machine learning to identify subtle CPDLC violations such as delayed acknowledgments, phase-inconsistent clearances, and ambiguous altitude expressions. To ensure transparency and auditability, detected anomalies are accompanied by explainable “why-flags” that explicitly link statistical alerts to protocol-level rationales and are presented through an operator-facing interface to support human decision-making. Using a fully synthetic CPDLC vignette dataset and a reproducible evaluation pipeline, we demonstrate conservative discriminative performance at a low false positive operating point, along with end-to-end processing latencies that indicate substantial headroom for incorporating more complex analysis without becoming a performance bottleneck. The framework is intentionally conservative and interpretable, and is designed to support future extensions incorporating more advanced learning-based techniques and additional protocol checks in safety-critical aviation communications.

I. INTRODUCTION

Controller–Pilot Data Link Communications (CPDLC) are a core component of modern air traffic management, enabling structured, text-based exchanges between air traffic controllers and flight crews that reduce voice congestion and mitigate readback errors in high-density airspace. As reliance on CPDLC continues to increase, so does the importance of ensuring not only operational efficiency, but also the integrity, correctness, and trustworthiness of the messages exchanged in safety-critical contexts.

Despite its operational benefits, CPDLC lacks built-in mechanisms for validating message correctness beyond basic syntactic compliance. Current systems generally assume that received messages are timely, semantically appropriate for the phase of flight, and unambiguous in their parameters. In practice, however, subtle protocol deviations (such as delayed acknowledgments, phase-inconsistent altitude instructions, or ambiguous clearance formats) can occur due to human error, system latency, or malicious interference. When left undetected, these issues may propagate into operational decisions with potentially serious safety consequences.

Prior research in aviation communication and automation has primarily focused on standardization, efficiency, and workload reduction, with increasing attention given to trustworthy

and human-centered AI decision-support tools for air traffic management [1]. Cybersecurity studies have further demonstrated that aviation datalink systems, including CPDLC, are vulnerable to spoofing and other forms of interference, highlighting the absence of robust integrity and authentication mechanisms at the protocol level [2]. However, comparatively little work has addressed the problem of detecting semantically incorrect or suspicious CPDLC messages in real time using protocol-aware and explainable methods suitable for safety-critical deployment.

This paper addresses this gap by presenting a protocol-aware AI framework for CPDLC monitoring that combines deterministic, contract-based rules with lightweight machine learning to support real-time anomaly detection. Rather than replacing human judgment, the framework is designed as a decision-support layer that surfaces potential issues through explicit, human-interpretable explanations. Detected anomalies are accompanied by “why-flags” that link each alert to a specific protocol rationale, enabling pilots and controllers to rapidly assess and respond to questionable communications.

Using a fully synthetic CPDLC vignette dataset and a reproducible evaluation pipeline, we demonstrate that this hybrid approach achieves strong discriminative performance while maintaining extremely low end-to-end latency. The resulting framework provides a practical and auditable foundation for enhancing CPDLC integrity and supports future research toward more advanced, certifiable AI-assisted safety systems in aviation communications.

II. METHODOLOGY

This work presents a protocol-aware framework for monitoring CPDLC messages that prioritizes interpretability, auditability, and real-time performance. Rather than relying on end-to-end machine learning, the system adopts a hybrid design in which deterministic protocol contracts provide primary validation while a lightweight statistical model supplies a secondary prioritization signal. This design reflects established guidance on trustworthy and certifiable AI in aviation, where transparency and bounded behavior are essential for safety-critical deployment [1], [3].

A. System Overview

The framework processes CPDLC-style message exchanges through a fixed pipeline consisting of parsing, contract rule evaluation, optional statistical scoring, and operator-facing explanation. Inputs are limited to text-based extracted CPDLC messages; no live operational datalink feeds or airline-identifying information are used. All processing is performed locally, and outputs are designed to support human decision-making rather than automated execution.

B. CPDLC Parsing and Schema Normalization

Incoming CPDLC messages are normalized into a frozen JSON schema that captures clearance identifiers, message role (controller or pilot), normalized action tokens, parameters (e.g., flight level), timestamps, acknowledgment state, flight phase, and metadata. Freezing the schema early enabled consistent downstream processing, reproducible dataset generation, and strict validation of message structure.

Normalized action tokens are drawn from the finite set {DESCEND_MAINTAIN, CLIMB_MAINTAIN, EXECUTE, READBACK, WILCO, STBY, UNABLE}¹. Acknowledgment state is represented separately via the `ack_state` field with values {WILCO, STBY, UNABLE, none}.

The ENROUTE phase subsumes cruise and non-terminal flight segments in this implementation (and is semantically equivalent to CRUISE for the purposes of rule evaluation).

Rejected messages are dropped prior to rule evaluation and machine learning inference and are logged with schema validation errors for auditability.

Messages are validated against the frozen schema prior to rule evaluation or machine learning inference. A message is rejected if required fields are missing, enumerated fields fall outside their allowed value sets, or timestamps are not valid ISO-8601 UTC. Messages with non-canonical altitude tokens are retained but flagged by the ambiguity contract (R3) rather than rejected. Rejected messages do not participate in downstream evaluation.

C. Contract Rules and Explainable Why-Flags

Protocol validation is performed using three contract-based rules derived from CPDLC operational semantics:

Each rule emits a deterministic verdict and an exact, human-readable “why-flag” string that explains the protocol rationale for the alert. These explanations are treated as first-class outputs and are preserved through the user interface and exported logs. This approach aligns with broader calls for transparent and human-centered automation in air traffic management [1].

D. Baseline Machine Learning Model

To complement deterministic rule checks, a lightweight binary classifier estimates whether a CPDLC exchange contains any protocol violation. The model uses TF-IDF features extracted from message text, optionally augmented with discrete protocol attributes (e.g., acknowledgment state, phase), and a

¹EXECUTE denotes an internal execute-gated control action in the synthetic controller and does not correspond to a literal CPDLC uplink message.

logistic regression classifier. This intentionally simple architecture was chosen to emphasize interpretability, reproducibility, and predictable behavior over raw predictive power, consistent with prior surveys of anomaly detection in aviation systems [4].

Model outputs are not used to override rule-based decisions. Instead, predicted probabilities serve as a supporting signal to prioritize operator attention when multiple exchanges are present. This separation reduces the risk of opaque automation while retaining the benefits of statistical pattern recognition.

TF-IDF features are extracted from the normalized message `text` field using unigrams and bigrams (`ngram_range=(1, 2)`), with English stop-word removal (`stop_words=english`), `min_df=1`, and `max_df=1.0`. Discrete protocol attributes (`role`, `ack_state`, and `phase`) are one-hot encoded and concatenated with the TF-IDF representation. Classification is performed using logistic regression with the default scikit-learn solver and a maximum of 1000 iterations; no class weighting is applied. The decision threshold τ is selected exclusively on the validation split to satisfy a target false positive rate constraint, and is frozen prior to evaluation on the held-out test set. The classifier operates at the message level. All message labels are inherited from vignette-level (cid-level) protocol rule outcomes, and vignette-level results are also reported for completeness.

E. Dataset Construction and Evaluation Protocol

All experiments were conducted using a fully synthetic CPDLC vignette dataset due to the sensitivity of operational datalink communications. Vignettes were generated as short, scenario-based exchanges (6–10 messages) with controlled injection of R1–R3 violations. Scenario-level train, validation, and test splits were enforced to prevent information leakage across datasets.

A small number of multi-violation vignettes were generated to reflect realistic exchanges in which multiple protocol violations may co-occur.

Message-level accounting. Table III reports dataset composition at the vignette level. Classifier training and evaluation, however, are performed at the message level, with all messages within a vignette inheriting the vignette’s label. Across all splits, vignettes contain between 6 and 10 messages, resulting in a larger number of message-level samples used for classification and confusion-matrix evaluation.

Vignette generation follows a fixed policy to ensure controlled coverage of protocol behaviors. Each vignette contains 6–10 messages and is labeled as NORMAL or containing one or more injected violations (R1, R2, R3). Dataset splits are performed strictly at the vignette level using a fixed random seed (1337), ensuring no message-level leakage between training, validation, and test sets. Raw counts by violation type and split are summarized in Table III.

Threshold selection for the classifier was performed exclusively on the validation set, with an explicit constraint on false positive rates to reflect operational safety considerations. Final

TABLE I
FROZEN CPDLC MESSAGE SCHEMA

Field	Type	Allowed Values / Format	Example
cid	string	Unique within vignette	SC042
role	enum	{CTL, PIL}	CTL
text	string	Raw message text	CTL DESCEND FL250
action	string	Normalized action token	DESCEND_MAINTAIN
params	object	Optional; altitude if canonical	{fl: FL250}
timestamp	string	ISO-8601 UTC (YYYY-MM-DDTHH:MM:SSZ)	2025-10-13T10:00:00Z
ack_state	enum	{WILCO, STBY, UNABLE, none}	none
phase	enum	{ENROUTE, CLIMB, APPROACH, UNKNOWN}	ENROUTE
meta	object	Freeform metadata (unused)	{source: synthetic}

TABLE II
PROTOCOL CONTRACT RULES AND EXPLAINABLE WHY-FLAGS

Rule	Constants	Trigger Condition	Why-Flag (Verbatim)
R1: Ack Timing	$T_{\text{warn}} = 45\text{s}$, $T_{\text{ack}} = 60\text{s}$	Execute-gated action without a WILCO acknowledgment within T_{ack}	Violation: No WILCO within 60s — DO NOT EXECUTE. Warning: STBY exceeded 45s; respond or query ATC.
R2: Phase–Altitude Sanity	FL200 threshold	phase = APPROACH and target altitude $\geq$ FL200	Atypical for current phase — VERIFY WITH ATC.
R3: Ambiguity Guard	Regex: $\text{^FL}\backslash\text{d}\{3\}\text{$}$	Missing or non-canonical altitude token	Altitude token ambiguous; expected FLxxx (e.g., FL250).
<i>Note:</i> Canonical altitude tokens consist of the prefix FL followed by exactly three digits (e.g., FL250).			

TABLE III
DATASET COMPOSITION BY VIOLATION TYPE AND SPLIT

Type	Train	Val	Test	Total
NORMAL	192	22	4	218
R1 (Ack Timing)	19	4	3	26
R2 (Phase–Altitude)	35	3	16	54
R3 (Ambiguity)	35	6	3	44
MULTI	7	1	10	18

TABLE IV
TOTAL MESSAGE COUNTS BY DATASET SPLIT

Split	Messages
Train	2331
Validation	296
Test	291
Total	2918

performance metrics were reported only on held-out test data. This conservative evaluation protocol addresses known pitfalls in aviation security and datalink research, where optimistic metrics can arise from unrealistic assumptions or data leakage [2].

F. Latency Measurement and Reproducibility

End-to-end and per-stage latency was measured using wall-clock timing (`time.perf_counter`) with 50 warm-up runs discarded, batch size set to one message, and 1000 measured repetitions per scenario to reflect real-time usage. All latency measurements were obtained on a CPU-only system (Intel i7-class processor, 32 GB RAM, Windows 11); no GPU acceleration was used. Measurements were reported as p50 and

p95 values, corresponding to the median and 95th-percentile latency, respectively.

Latency was measured across three pipeline stages: CPDLC message parsing, scenario-level protocol rule evaluation, and classifier inference. Classifier timing reflects inference-only cost (`predict_proba` on a minimal dummy feature vector), intentionally excluding TF-IDF and one-hot feature extraction and vectorization; reported classifier latency therefore represents a lower-bound estimate. End-to-end latency measurements include parsing, rule evaluation, and per-message classifier inference, and therefore represent the operationally relevant performance metric.

All experiments are fully reproducible from a clean repository clone. Dataset generation, model training, scoring, latency measurement, and API execution are controlled by documented scripts with fixed random seeds. No external databases, cloud services, or proprietary data sources are required.

III. RESULTS AND DISCUSSION

This section presents the empirical evaluation of the proposed CPDLC monitoring framework, focusing on the performance of the baseline machine learning component, its interaction with deterministic protocol rules, and measured system latency. All results are reported using the final, leakage-free dataset and evaluation protocol described in Section II.

A. Classification Performance

The baseline classifier was evaluated on a held-out test set consisting of scenario-level CPDLC vignettes generated under controlled conditions. Model selection and threshold calibration were performed exclusively on the validation split, with the explicit objective of constraining false positives in line

with operational safety requirements. The selected operating threshold τ was frozen prior to test evaluation. These message-level figures reflect a deliberately conservative operating point tuned to minimize false alarms; deterministic protocol contracts remain the primary detection mechanism. All metrics reported in Table V and Figures 3–4 are computed at the message level, consistent with the operating granularity of the classifier (see Table VII).

Table V summarizes classification performance on the held-out test set. The reported metrics correspond to a deliberately conservative operating point selected to strongly suppress false positives, resulting in intentionally limited recall at the chosen threshold. This operating point reflects the intended use of the classifier as a secondary prioritization signal, where missed detections are mitigated by deterministic protocol rules and statistical scores are not treated as autonomous decisions. Overall discriminative performance is summarized using ROC-AUC and PR-AUC, which denote the area under the receiver operating characteristic (ROC) and precision–recall (PR) curves, respectively, with AUC referring to the area under the corresponding curve.

At the message level, PR-AUC is high (0.94) while ROC-AUC is modest (0.63) because the test split is highly imbalanced (e.g., 259 positives vs 32 negatives) and we chose a conservative operating threshold ($\tau = 0.458$) to cap FPR—yielding high precision at the cost of recall.

TABLE V
CLASSIFICATION PERFORMANCE ON HELD-OUT TEST SET
(MESSAGE-LEVEL)

Metric	Value
ROC-AUC	0.630
PR-AUC	0.942
Decision threshold (τ)	0.458
TPR @ τ	0.266
FPR @ τ	0.000
True Positives (TP)	69
False Positives (FP)	0
True Negatives (TN)	32
False Negatives (FN)	190

Note: Metrics are reported at the message level. TPR = TP/(TP+FN) = 69/259 = 0.266. FPR = FP/(FP+TN) = 0/32 = 0.000, with an exact binomial upper bound ≈ 0.010 at 95% confidence (Clopper–Pearson).

Figures 1, 3, and 4 show the corresponding message-level confusion matrix, ROC (receiver operating characteristic) curve, and precision–recall (PR) curve generated from the frozen evaluation artifacts. Unlike earlier pilot experiments that exhibited artificially perfect separation due to dataset leakage, the corrected curves reflect a more realistic classification regime under strict scenario-level splitting.

To assess whether the classifier’s prioritization signal varies across different protocol violations, we further report message-level performance stratified by rule (R1–R3) at the fixed operating threshold τ (Table VI).

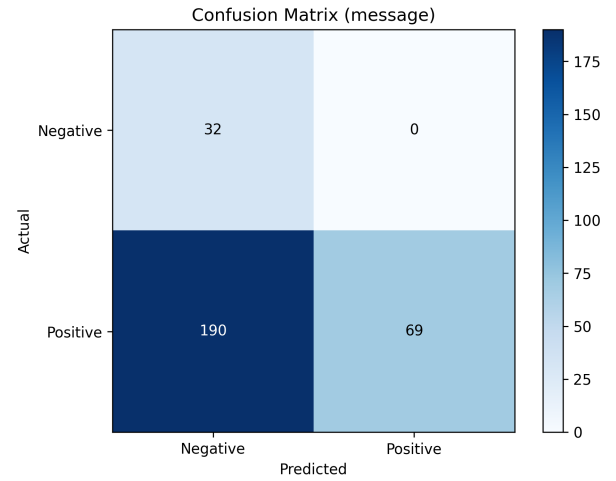


Fig. 1. Message-level confusion matrix for the baseline classifier.

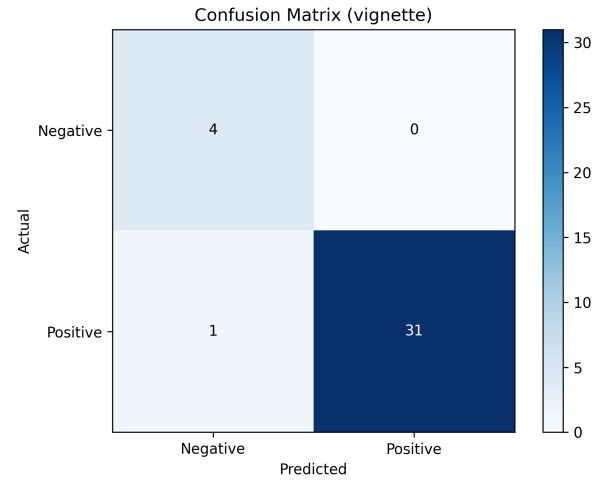


Fig. 2. Vignette-level confusion matrix for the baseline classifier.

B. Rule-Based vs Statistical Signal Ablation

Tables VII and VIII illustrate the overlap between deterministic protocol rule violations and classifier alerts on the held-out test set at the vignette level. Table VII reports raw vignette counts, while Table VIII presents the corresponding rates for interpretability. All rule-flagged violations exceed the classifier decision threshold at the selected operating point, indicating strong alignment between the statistical signal and deterministic contracts. The absence of model-positive outcomes for rule-passing vignettes reflects the intentionally conservative threshold selection and supports the role of the classifier as a prioritization signal rather than a primary detection mechanism.

C. Representative Failure Cases

To provide qualitative insight into system behavior, we examined representative false negatives and false positives on the held-out test set. The examples below illustrate typical

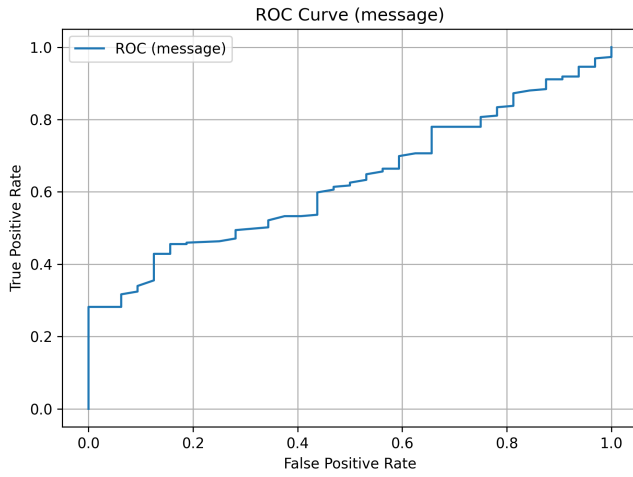


Fig. 3. ROC curve for the baseline classifier (message-level).

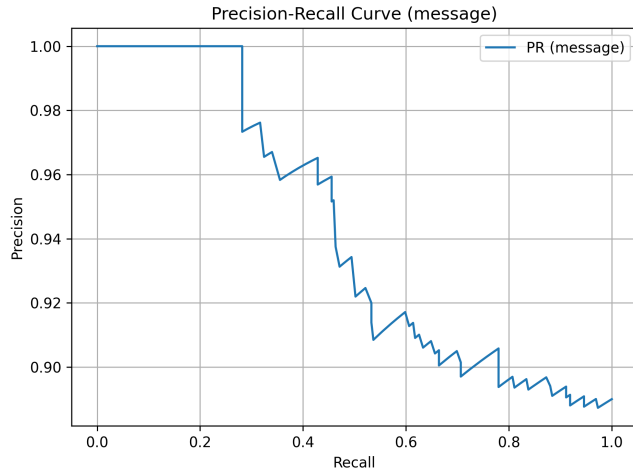


Fig. 4. Precision-recall curve for the baseline classifier (message-level).

TABLE VI
PER-RULE MESSAGE-LEVEL METRICS AT FIXED THRESHOLD τ

Rule	TPR	FPR	TP	FP	FN	TN
R1 (Ack Timing)	0.511	0.114	46	23	44	178
R2 (Phase-Altitude)	1.000	0.187	18	51	0	222
R3 (Ambiguity)	0.800	0.227	4	65	1	221

Note: Metrics are reported at the message level. For each rule, $TPR = TP/(TP+FN)$ and $FPR = FP/(FP+TN)$. Exact 95% binomial confidence intervals (Clopper-Pearson) are: R1: $TPR = 0.511$ [0.404, 0.617], $FPR = 0.114$ [0.074, 0.164]; R2: $TPR = 1.000$ [0.815, 1.000], $FPR = 0.187$ [0.143, 0.238]; R3: $TPR = 0.800$ [0.284, 0.995], $FPR = 0.227$ [0.180, 0.280].

failure modes and clarify the distinct roles of deterministic protocol rules and the statistical classifier.

- **False negative (classifier prioritization, R3):** A vignette containing an altitude ambiguity (non-canonical token) was correctly flagged by the ambiguity contract but received a classifier score below the decision threshold, resulting in reduced statistical prioritization despite a deterministic violation (e.g., “CTL DESCEND FL25”;

TABLE VII
OVERLAP BETWEEN RULE-BASED VIOLATIONS AND CLASSIFIER ALERTS ON THE TEST SET (VIGNETTE-LEVEL COUNTS)

Rule Outcome	Model Positive	Model Negative	Total
Rule Fail	31	1	32
Rule Pass	0	4	4
Total	31	5	36

TABLE VIII
OVERLAP BETWEEN RULE-BASED VIOLATIONS AND CLASSIFIER ALERTS ON THE TEST SET (VIGNETTE-LEVEL RATES)

Rule Outcome	Model Positive (%)	Model Negative (%)
Rule Fail	96.9	3.1
Rule Pass	0.0	100.0

violates $\hat{FL}\{3\}$).

- **False negative (classifier prioritization, R1):** A delayed acknowledgment sequence near the R1 timing boundary triggered a blocking acknowledgment-timing violation but was assigned a low classifier probability due to sparse lexical indicators in the message text (e.g., “CTL CLIMB FL230” $\rightarrow$ “PIL EXECUTE”; no WILCO within 60s).
- **False positive (classifier only):** A syntactically valid descent clearance with unusual phrasing triggered a classifier alert despite passing all deterministic protocol contracts, reflecting conservative sensitivity to rare n-gram patterns (e.g., “CTL DESCEND FL210”; rules R1–R3 pass).
- **False positive (classifier only):** A normal approach-phase exchange containing atypical but valid altitude language was flagged by the model while remaining compliant with all rule checks (e.g., “CTL DESCEND FL190” with *phase=APPROACH*; below the R2 threshold FL200).

D. Latency and System Performance

End-to-end latency measurements demonstrate that the complete processing pipeline, including raw-text parsing, scenario-level rule evaluation, feature extraction (TF-IDF and one-hot encoding), and classifier inference, operates well below real-time constraints. Reported end-to-end latency reflects the full pipeline cost rather than inference-only timing, providing a realistic upper bound on operational overhead.

TABLE IX
PIPELINE LATENCY BREAKDOWN (CPU-ONLY, WALL-CLOCK TIMING)

Stage	p50 (ms)	p95 (ms)
Parsing	0.061	0.081
Rule Evaluation	0.349	0.470
Feature Extraction	0.592	0.746
Classifier Inference	0.066	0.094
End-to-End	1.155	1.412

Note: Feature extraction includes TF-IDF vectorization and one-hot encoding. For reference, inference-only latency excluding feature extraction yields $p95 = 0.595$ ms and is reported separately to illustrate lower-bound model cost.

Measured end-to-end latency exhibits substantial headroom relative to a conservative 200 ms operational budget ($p95 = 1.41 \text{ ms} \ll 200 \text{ ms}$).

E. Interpretation and Operational Implications

Although the classifier demonstrates useful aggregate performance, its outputs are not intended to serve as autonomous decisions. Instead, predicted probabilities act as a prioritization signal alongside deterministic rule-based checks. All alerts surfaced to operators are accompanied by explicit protocol-level why-flags, ensuring that statistical scores are grounded in interpretable, auditable rationales.

The small number of false positives observed at the chosen threshold reflects a deliberate trade-off favoring safety. In operational contexts, unnecessary alerts can degrade trust and increase workload, whereas missed detections are mitigated by the presence of deterministic contract rules. This layered approach allows statistical learning to enhance, rather than replace, protocol-aware validation.

An operator-facing web interface was implemented to demonstrate end-to-end usage of the framework. The interface allows users to upload CPDLC message logs or generate synthetic test exchanges, view parsed message fields and protocol rule outcomes, inspect generated why-flags, and export analysis results in JSON format. The interface is served by a lightweight local API backend and is intended solely as a demonstration and testing tool, illustrating how protocol-aware explanations can be surfaced to human operators rather than as a production-ready system. The interface was not used for quantitative evaluation and serves solely as a demonstrative artifact.

F. Limitations

All results reported in this study are derived from synthetic CPDLC vignettes, necessitated by the sensitivity and restricted availability of operational datalink data. As a result, reported ROC-AUC, PR-AUC, and threshold-based metrics reflect performance on controlled, single-violation synthetic scenarios and should not be interpreted as direct indicators of operational or real-world CPDLC performance. While care was taken to generate realistic scenarios and eliminate evaluation leakage, future work should assess generalization using larger and more diverse datasets where permissible. Additionally, further threshold tuning and feature refinement may reduce the remaining false positives without sacrificing detection sensitivity.

Overall, these results demonstrate that a conservative, protocol-aware hybrid architecture can achieve strong anomaly detection performance while preserving interpretability, auditability, and real-time responsiveness.

IV. FUTURE WORK

The proposed framework is intentionally conservative, prioritizing protocol awareness, interpretability, and auditability over model complexity. While the current implementation demonstrates the effectiveness of combining deterministic rule

checks with lightweight machine learning, several promising research directions could substantially extend its capabilities while preserving its suitability for safety-critical aviation environments.

One natural extension is the integration of locally deployed large language models (LLMs) to enhance semantic and contextual analysis of CPDLC message exchanges. Recent work has shown that domain-adapted LLMs can effectively model aviation-specific language and safety reports, enabling higher-level pattern recognition and anomaly detection beyond surface-level syntax [5], [6]. Within the proposed framework, LLMs could serve as auxiliary analyzers that assess multi-message consistency, detect unusual phrasing or intent, and surface latent risk indicators that are not easily captured by fixed rules or n-gram features. Running such models locally would avoid external data dependencies and support confidentiality and controllability in operational settings [7].

A second avenue for future research involves strengthening message authenticity and integrity through cryptographic verification. Prior studies have demonstrated that aviation datalink systems remain vulnerable to spoofing and interference at the protocol level [2], [8]. The current rule engine could be extended to incorporate digital signature validation as an additional contract layer, with new rules and explainable why-flags explicitly indicating missing, invalid, or mismatched signatures. Such an extension would enable operators to distinguish between semantic protocol violations and potential integrity or authenticity breaches, further enhancing cybersecurity situational awareness.

Beyond CPDLC, the underlying design principles of protocol-aware parsing, contract-based validation, and explainable alerting may generalize to other aviation data links. In particular, extending this approach to Automatic Dependent Surveillance–Broadcast (ADS-B) communications represents a challenging but potentially impactful direction. Recent research has explored the application of machine learning and large language models to ADS-B data for trajectory estimation and anomaly analysis [9]. Applying similar concepts in real-time would require scalable ingestion pipelines, domain-specific validation contracts, and careful consideration of data volume, timing constraints, and heterogeneity across avionics equipment. While substantially more complex than CPDLC, such an extension could contribute to broader efforts aimed at improving the trustworthiness and resilience of aviation surveillance systems.

Finally, future work may include human-in-the-loop evaluations to study how pilots and air traffic controllers interpret and act upon why-flags in realistic scenarios, as well as investigations into certification-aligned deployment pathways. Together, these directions highlight how the proposed framework can serve as a foundation for continued research into secure, explainable, and protocol-aware AI systems for aviation communications.

V. CONCLUSION

This paper presented a protocol-aware AI framework for monitoring Controller–Pilot Data Link Communications (CPDLC) that combines deterministic contract-based validation with lightweight machine learning to support real-time anomaly detection. By grounding the system in a frozen JSON schema, formally defined rules, and explicit why-flags, the framework emphasizes transparency, auditability, and human interpretability, which are key requirements for safety-critical aviation systems.

The results demonstrate that even a simple statistical baseline, when paired with protocol-aware rules, can effectively distinguish between compliant and violating CPDLC message exchanges while maintaining extremely low end-to-end latency. Rather than relying on opaque models, the proposed approach surfaces clear protocol rationales alongside probabilistic indicators, enabling pilots and controllers to make informed decisions without displacing human judgment.

Importantly, the framework was designed with reproducibility and extensibility in mind. All datasets are synthetic and scenario-based, evaluation pipelines are deterministic, and performance and latency metrics are explicitly reported. This makes the system suitable both as a research artifact and as a foundation for future work exploring more advanced learning-based components, cryptographic integrity checks, and broader applications to other aviation data links.

Overall, this work illustrates how protocol-aware design and explainable AI techniques can be integrated into practical tools for aviation communications monitoring. By prioritizing clarity, conservatism, and alignment with operational constraints, the proposed framework offers a viable pathway toward more trustworthy AI-assisted safety systems in modern aviation.

ARTIFACTS AVAILABILITY

All code, datasets, and evaluation scripts used in this paper are available in a version-controlled repository. To preserve the integrity of the review process, the repository is currently private and will be made publicly available upon acceptance. All results reported in this paper were generated from a frozen repository state at commit `f3a3080`, which is documented in `docs/REPRO.md`.

ACKNOWLEDGMENTS

Special thanks to Prof. Amir Toliyat for advising this work, Dr. Maryam Vatankhah for feedback on methodology, and Capt. Christopher Fuller for operational subject-matter input.

REFERENCES

- [1] “Ai in air traffic management: Bringing intelligent and trustworthy automation to aviation,” European Commission, Tech. Rep., 2022. [Online]. Available: https://publications.europa.eu/resource/cellar/b555ad1b-5e57-11ed-92ed-01aa75ed71a1.0001.01/DOC_1
- [2] S. Sathaye *et al.*, “On the implications of spoofing and jamming aviation datalink applications,” in *Proceedings of the ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec)*, 2022. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3564625.3564651>
- [3] L. Bello *et al.*, “Towards certifiable ai in aviation: Landscape, challenges, and opportunities,” *arXiv preprint arXiv:2406.11318*, 2024. [Online]. Available: <https://arxiv.org/pdf/2406.11318.pdf>
- [4] L. Basora *et al.*, “Recent advances in anomaly detection methods applied to aviation,” *Aerospace*, vol. 6, no. 11, p. 117, 2019. [Online]. Available: <https://www.mdpi.com/2226-4310/6/11/117>
- [5] L. Wang, J. Chou, X. Zhou, A. Tien, and D. M. Baumgartner, “Aviationgpt: A large language model for the aviation domain,” *arXiv preprint arXiv:2311.17686*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.17686>
- [6] A. Tikayat Ray, A. P. Bhat, R. T. White, V. M. Nguyen, O. J. Pinon Fischer, and D. N. Mavris, “Examining the potential of generative language models for aviation safety analysis,” *Aerospace*, vol. 10, no. 9, p. 770, 2023. [Online]. Available: <https://www.mdpi.com/2226-4310/10/9/770>
- [7] Y. Liu, “Large language models for air transportation: A critical review,” *Journal of Air Transport Research Society*, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2941198X24000356>
- [8] “Cyber security in aviation automationsprogrammet ii del ii,” DiVA Portal, Tech. Rep., 2024. [Online]. Available: <http://www.diva-portal.org/smash/get/diva2:1994286/FULLTEXT01.pdf>
- [9] Q. Zhang, “An exploratory assessment of llms’ potential for flight trajectory estimation using ads-b data,” *Mathematics*, vol. 13, no. 11, p. 1775, 2025. [Online]. Available: <https://www.mdpi.com/2227-7390/13/11/1775>